CASPO: Learned Prefix Value Modeling for Stepwise Reasoning Optimization

TODO: Authors

Draft: April 26, 2026

Abstract

This draft describes the current CASPO codebase and experimental stack. CASPO is a full-model reinforcement learning method for mathematical reasoning that replaces VinePPO's Monte Carlo prefix rollouts with a learned IPVRM prefix value model. The repository now supports four methods in a common trainer: terminal-reward PPO, grouped-relative GRPO, VinePPO with $K = 9$ prefix rollouts, and CASPO with offline plus online prefix value learning. The current paper-faithful target is Rho-1B on MATH with 512 responses per PPO outer step, vLLM rollout generation, CUDA-IPC weight sync, and one H100 per method. This manuscript intentionally reports infrastructure and speed-probe measurements only; accuracy results should be added after the full training and evaluation runs complete.

1 Introduction

Sparse final-answer rewards make reinforcement learning for long mathematical reasoning difficult. VinePPO addresses this by branching from intermediate reasoning prefixes and sampling Monte Carlo continuations, yielding better credit assignment at the cost of many extra generations [2]. CASPO keeps the same stepwise PPO structure but replaces those online prefix rollouts with a learned prefix value model inspired by IPVRM [1].

The intended comparison is not only algorithmic but also systems-oriented: given the same policy, data, verifier, rollout budget, and PPO update shape, how much step-level signal can be recovered without VinePPO's repeated suffix expansion? The current repository therefore implements PPO, GRPO, VinePPO, and CASPO in one stack so wall-clock and accuracy can be measured under the same vLLM and trainer infrastructure.

2 Related Work

PPO. PPO optimizes a clipped policy-gradient surrogate to prevent excessively large policy updates [4]. In this codebase, PPO uses terminal verifier rewards and broadcasts a sequence-level advantage over response tokens before applying the clipped language-model PPO objective with a KL penalty to a frozen reference policy.

VinePPO. VinePPO estimates values at intermediate reasoning states by sampling continuations from each prefix and grading the completed trajectories [2]. The method avoids relying on a learned critic for complex reasoning credit assignment, but its cost scales with the number of prefixes, continuations, and reward evaluations.

IPVRM. IPVRM learns prefix-conditioned values from trajectory-level outcome labels, then uses temporal-difference differences between adjacent prefix values as process-level signals [1]. The CASPO implementation uses this prefix-value view as the replacement for VinePPO's Monte Carlo prefix value estimator. Unlike the IPVRM paper's LoRA setting, the current code performs full-model value updates, so the online value learning rate is set conservatively to 10^{-6}.

GRPO. GRPO was introduced in DeepSeekMath as a PPO variant that uses grouped samples and relative rewards while avoiding a learned value network [5]. In this repository, GRPO is implemented as a terminal-reward baseline: for each prompt, the $G = 8$ sampled responses are normalized relative to their group and then trained with the same clipped PPO loss as the other methods.

vLLM. The stack uses vLLM for high-throughput generation and prefix-cache reuse [3]. The current Rho-1B setup uses vLLM's CUDA-IPC weight-transfer path for trainer-to-vLLM synchronization, avoiding checkpoint-to-disk reloads after every policy update.

3 Method

3.1 Step Segmentation

Each generated response is segmented into reasoning steps. The current MATH configuration uses the LaTeX-aware splitter ported from VinePPO rather than a simple newline delimiter. Let s_t denote the prefix at the start of step t and let $R(x, y) \in \{0, 1\}$ be the final verifier reward.

3.2 CASPO Prefix Values

CASPO uses an IPVRM-style prefix value model initialized from the same SFT checkpoint family as the policy. Given response tokens y_i, the code constructs prefix values from cumulative log-ratios:

$$V_\phi(s_t) = \beta \sum_{i<t} \log \frac{\pi_\phi(y_i \mid x, y_{<i})}{\pi_{\text{ref}}(y_i \mid x, y_{<i})}. \tag{1}$$

The value model is first trained offline with terminal correctness labels and then updated online during policy training with ADB/DLW enabled. The current Rho-1B MATH value checkpoint lives at:

`/mnt/nvme_tmp/jason_caspo/caspo_rho1b_math/value_final`.

Its training summary reports 5960 train rollouts, 664 validation rollouts, three epochs, best validation loss 0.8037, and last validation accuracy 0.9484.

3.3 Step TD Advantage

CASPO and VinePPO both use step-level temporal-difference advantages:

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{2}$$

where $\gamma = 1$ and r_t is zero except on the terminal reasoning step, where $r_T = R(x, y)$. CASPO obtains V from the learned prefix value model; VinePPO obtains V by sampling $K = 9$ continuations from nonterminal prefixes and averaging their verifier rewards.

The current CASPO configuration standardizes valid step advantages over the whole rollout batch:

Table 1: Current Rho-1B MATH configuration.

Field	Value
Base policy	`realtreetune/rho-1b-sft-MATH`
Train data	`DigitalLearningGmbH/MATH-lighteval`, train split
Eval data	`HuggingFaceH4/MATH-500`, test split
Prompt template	VinePPO MATH task template
Max response length	1024 tokens
Group size	8 responses per prompt
Prompts per outer step	64
Responses per outer step	512
PPO minibatch	64 responses
PPO epochs per rollout	2
Policy LR	10^{-6}
Online value LR	10^{-6}
CASPO advantage transform	`value`
PPO clip	0.2
KL coefficient	10^{-4}
Rollout sampling	temperature 0.6, top-p 0.9
Training length	1000 outer steps
Checkpoints	`step_250`, `step_500`, `step_750`, `final`

`standardize_step_advantage=true`, `standardize_advantage_scope=batch`.

Thus, for the Rho-1B MATH run, advantages are whitened across all valid steps from all 512 responses in the outer PPO step, not separately per prompt group.

The default CASPO run computes TD on the direct IPVRM value. The code also supports two pre-normalization ablations that transform the prefix values before the TD difference: `prob` uses $\sigma(V)$ and `logprob` uses $\log \sigma(V)$. The terminal verifier reward term is unchanged in all three variants.

3.4 PPO Objective

All four methods ultimately use the same clipped PPO token objective:

$$\mathcal{L}_{\text{clip}}(\theta) = -\mathbb{E}_t \left[\min \left(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) \right], \tag{3}$$

where $\rho_t(\theta)$ is the token probability ratio between the current policy and the rollout policy. The implementation also applies a KL penalty to the frozen reference policy using the $k3$ estimator with coefficient 10^{-4}.

4 Current Experimental Configuration

The main experiment uses `configs/caspo_rho1b_math.yaml`. Table 1 summarizes the current paper-faithful Rho-1B MATH settings.

Table 2: Rho-1B MATH speed probe with paper-faithful 512-response outer steps.

Method	Step time	Rollout	Value/MC	Policy
PPO	90 s	4 s	0 s	74 s
GRPO	92 s	4 s	0 s	76 s
CASPO	141 s	4 s	55 s	69 s
VinePPO $K = 9$	237 s	4 s	152 s	69 s

5 Training and Inference Stack

The production Rho-1B run uses the `scalable` conda environment and writes data, logs, checkpoints, and Hugging Face caches under `/mnt/nvme_tmp/jason_caspo`. The launch command is:

```
RUN_TAG=paper512_seed0 GPU_LIST="4 5 6 7" WANDB_MODE=offline \
  ./scripts/launch_rho1b_parallel.sh
```

Each method gets one H100: PPO, CASPO, GRPO, and VinePPO are mapped to the four listed GPUs in that order.

The vLLM rollout backend is enabled for all methods. For Rho-1B, the code uses CUDA-IPC weight synchronization:

`vllm_weight_sync_backend=ipc`.

Recent IPC probes show sync around 0.3–0.4 seconds per step, so the current bottleneck is policy/value computation rather than weight transfer.

The embedded vLLM path also primes the AsyncLLM output handler on the repository's persistent event loop. This is required for the current vLLM V1 runtime, where constructing AsyncLLM outside a running event loop can otherwise stall on EngineCore metadata responses.

6 Speed Probe

Table 2 reports the latest three-step paper-faithful speed probe. These are systems measurements, not final accuracy results. The probe used one H100 80GB per method, 512 responses per outer step, `save_every=0`, and vLLM IPC sync.

PPO uses terminal rewards, GRPO uses grouped terminal rewards, CASPO adds a learned value forward/update phase, and VinePPO's value column is dominated by MC prefix continuation scoring.

The implied 1000-step wall-clock estimates are approximately 25 hours for PPO, 26 hours for GRPO, 39 hours for CASPO, and 66 hours for VinePPO $K = 9$. Running all four methods in parallel on GPUs 4–7 is gated by VinePPO, so the full sweep should take roughly 66–72 hours before optional evaluation overhead.

7 Evaluation Plan

Periodic evaluation should be a cheap sample eval, not the full benchmark suite. The launcher supports evaluating a saved checkpoint with:

```
RUN_TAG=paper512_seed0 CKPT_SUBDIR=step_250 \
EVAL_BENCHMARKS=math500 EVAL_LIMIT=100 EVAL_K=8 \
EVAL_GPU_LIST="4 5 6 7" ./scripts/launch_eval_all.sh
```

Recommended periodic checkpoints are `step_250`, `step_500`, `step_750`, and `final`. Full evaluation should be run only on `final` unless intermediate curves become necessary for analysis. CASPO advantage ablations can be launched with `scripts/launch_rho1b_caspo_ablations.sh`; by default this schedules only the two extra `prob` and `logprob` variants because `value` is the main CASPO run. The frozen-RM ablation uses `scripts/launch_rho1b_caspo_frozen_rm.sh`, keeping IPVRM scoring while setting `update_value_during_policy=false`. For faster VinePPO on Rho-1B, the repository also includes `scripts/launch_rho1b_vineppo_ddp2.sh`, a two-GPU replicated DDP launcher with one rank-local vLLM engine and IPC weight sync per rank. It launches one Python process per physical GPU so each trainer rank and vLLM engine share the same single visible CUDA device. Its default shape is 32 prompts per rank with group size 8, preserving the 512-response global outer step.

8 Limitations and Open Engineering Questions

CASPO depends on the quality and calibration of the learned prefix value model. Online updates mitigate distribution shift but can also drift if the learning rate is too high; the current full-model setting therefore uses a conservative online value learning rate. The current Rho-1B setup is optimized for one H100 per method. Scaling to 7B full fine-tuning likely requires FSDP and currently falls back to checkpoint-based vLLM sync until an exact-runtime NCCL or in-memory sync path is implemented.

The latest speed probe indicates that rollout generation and trainer-to-vLLM sync are no longer the main bottlenecks. PPO/GRPO are dominated by policy forward/backward time, CASPO adds a stable value-model phase, and VinePPO is dominated by Monte Carlo prefix evaluation. Further speedups would require changing the experimental contract: fewer PPO epochs, LoRA/PEFT instead of full-model updates, fewer VinePPO rollouts, capped MC continuation lengths, or multi-GPU training per method.

9 Conclusion

The current CASPO repository is ready to run a four-method, paper-faithful Rho-1B MATH comparison using a shared full-model RL stack. The key experimental question remains empirical: whether learned IPVRM prefix values can match or exceed VinePPO's rollout-derived credit assignment at lower wall-clock cost, without introducing reward-model drift or unstable policy updates.

References

[1] Shiping Gao, Hongzhan Chen, Xiaojun Quan, Qifan Wang, and Lifu Huang. Unleashing implicit rewards: Prefix-value learning for distribution-level optimization. *arXiv preprint arXiv:2604.13197*, 2026. URL `https://arxiv.org/abs/2604.13197`.

[2] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning

through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024. URL https://arxiv.org/abs/2410.01679.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[5] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, Daya Guo, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.